Caching Strategies

Cache eviction policies determine which items to remove when the cache reaches its capacity limit. Choosing the right policy significantly impacts cache hit rates and application performance.

Common Eviction Policies

1. **LRU (Least Recently Used)**

   Removes the item that hasn't been accessed for the longest time.

   How it works: Maintains a timestamp or ordering of accesses. When eviction is needed, removes the item with the oldest access time.

   Best for: General-purpose caching with temporal locality (recently accessed items are likely to be accessed again)

   Example: If you access items A, B, C, D, then A again, the access order is B, C, D, A. If cache is full, B gets evicted first.

   Pros: Excellent for many real-world workloads, intuitive behavior
   Cons: Doesn't account for access frequency, can be fooled by one-time scans of large datasets

2. **LFU (Least Frequently Used)**

   Removes the item accessed the fewest times.

   How it works: Maintains a counter for each cache entry. Evicts the entry with the lowest count.

   Best for: Workloads where some items are consistently popular while others are rarely accessed

   Example: Item A accessed 100 times, B accessed 50 times, C accessed 5 times. C gets evicted first.

   Pros: Protects frequently used items, good for skewed access patterns
   Cons: Old popular items may never be evicted even if no longer relevant, new items easily evicted

3. **FIFO (First In, First Out)**

   Removes the oldest item in the cache, regardless of access patterns.

   How it works: Items are evicted in the order they were added to the cache.

   Best for: Simple caching scenarios where access patterns are random or implementation simplicity is paramount

   Pros: Simple to implement, predictable behavior, low overhead
   Cons: Ignores usage patterns entirely, poor hit rates compared to LRU/LFU

4. **LIFO (Last In, First Out)**

   Removes the most recently added item.

   How it works: The newest item gets evicted first.

   Best for: Rare in practice; occasionally useful for specific stack-like access patterns

   Pros: Very simple to implement
   Cons: Usually performs poorly for caching; mostly theoretical

5. **Random Replacement -** Randomly selects an item to evict.

   How it works: Uses a random number generator to pick which entry to remove.

   Best for: When implementation simplicity is critical and performance requirements are modest

   Pros: Zero overhead, no metadata needed, surprisingly competitive performance
   Cons: Unpredictable, can evict hot items by chance

6. **MRU (Most Recently Used)**

   Removes the most recently accessed item.

   How it works: Opposite of LRU—evicts the item that was just accessed.

   Best for: Sequential scan patterns where recently accessed items won't be accessed again

   Example: Reading through a large file sequentially—once you've read a block, you won't need it again

   Pros: Excellent for specific sequential access patterns
   Cons: Terrible for general workloads with temporal locality

Advanced Eviction Policies

LRU-K - Tracks the last K accesses for each item. Evicts based on the timestamp of the Kth most recent access.

Advantage: More resistant to scans; LRU-2 is particularly popular as it balances recency and frequency.

ARC (Adaptive Replacement Cache) - Dynamically balances between recency (LRU) and frequency (LFU) by maintaining two lists and adapting based on workload.

Advantage: Self-tuning, excellent performance across diverse workloads
Disadvantage: More complex, may have patent concerns

SLRU (Segmented LRU) - Divides cache into protected and probationary segments. New items start in probationary; items accessed twice move to protected.

Advantage: Protects frequently accessed items from one-time scans
Disadvantage: Requires tuning segment sizes

TTL (Time-To-Live) Based - Items expire after a fixed time period regardless of access.

Advantage: Guarantees freshness, simple to reason about
Disadvantage: Not based on capacity—can waste space or evict items prematurely

Size-Aware Eviction - Considers the size of cached items, not just count. May evict larger items to make room for multiple smaller ones.

Advantage: Better space utilization for variable-sized objects
Disadvantage: More complex cost-benefit calculations

Performance Comparison

For typical web application workloads (ranked by hit rate):

1. **ARC** - Best adaptive performance
2. **LRU** - Excellent general-purpose choice
3. **LRU-2** - Better than LRU for mixed workloads
4. **LFU** - Good for highly skewed access patterns
5. **Random** - Surprisingly decent baseline
6. **FIFO** - Simple but suboptimal
7. **LIFO/MRU** - Poor for typical caching